Valence Alignment in Human-AI Story Co-Creation

Analysis of Field Data and AI-AI Baseline Comparison

PENPAL Project

2025-11-29

Contents

0.1 Summary . . . 2

1 1. Data Loading and Preprocessing 2
1.1 1.1 Load Data . . . 2
1.2 1.2 Data Preprocessing . . . 3
1.3 1.3 Compute Lagged Variables and Cosine . . . 4

2 2. Descriptive Statistics 5
2.1 2.1 Token Count Statistics . . . 5
2.2 2.2 Valence Distribution Statistics . . . 6

3 3. Valence Distribution Analysis 7
3.1 3.1 Combined Valence Distribution Plot . . . 7

4 4. Valence Alignment Analysis 9
4.1 4.1 Per-Story Correlation (Same Turn) . . . 9
4.2 4.2 Per-Story Correlation (Previous Turn) . . . 10
4.3 4.3.1 All turns story correlation (same turn) . . . 11
4.4 4.3.2 All turns story correlation (Previous Turn) . . . 12
4.5 4.3.3 Probe Sentiment Narrative Development . . . 13
4.6 4.4.1 Alignment Comparison Plot . . . 14
4.7 4.4.2 Comparison: Per-Story vs Aggregate Correlations . . . 15
4.8 4.5 Mixed Effects Model: Random Intercepts per Story . . . 17
4.9 4.6 Statistical Tests: Alignment Differences . . . 20

5 5. Summary and Conclusions 22
5.1 5.1 Key Findings Summary . . . 22

6 6. Supplementary: Interaction Effects 23
6.1 6.1 Starter Interaction Effects . . . 23
6.1.1 6.1.1 Same-Turn Alignment by Starter . . . 23
6.1.2 6.1.2 Previous-Turn Alignment by Starter . . . 23
6.1.3 6.1.3 Per-Story Correlations by Starter . . . 24
6.2 6.2 LLM Type Interaction Effects . . . 25
6.2.1 6.2.1 Same-Turn Alignment by LLM Type . . . 25
6.2.2 6.2.2 Previous-Turn Alignment by LLM Type . . . 26
6.2.3 6.2.3 Per-Story Correlations by LLM Type . . . 27
6.3 6.3 Valence Distribution by Conditions . . . 28
6.3.1 6.3.1 Valence by Starter . . . 28

6.3.2 6.3.2 Valence by LLM Type 29
6.4 7. Session Information 29

0.1 Summary

- Emotional alignment is asymmetric: the model reliably aligns to the user (User → AI), while humans do not mirror the preceding AI tone.
- AI–AI interactions show higher, bidirectional alignment driven by symmetric context access.
- Field data shows greater variance in valence and weaker overall coordination compared to the simulated dataset.
- Participant-level modelling reveals a consistent "rubber-band" effect, where early divergence returns toward baseline in later stages.
- Overall, affective dynamics in the field are primarily model-driven rather than mutually convergent.

1 1. Data Loading and Preprocessing

1.1 1.1 Load Data

```r
# Load dyadic sentiment scores

df_field <- read_parquet(here("data/TEXT/processed/dyadic_sentiment_scores.parquet"))
df_simulated <- read_parquet(here("data/Berlin/processed/dyadic_sentiment_scores_simulated.parquet"))

# Display structure
cat("Field data dimensions:", dim(df_field), "\n")
```

```
## Field data dimensions: 820 16
```

```r
cat("Simulated data dimensions:", dim(df_simulated), "\n")
```

```
## Simulated data dimensions: 30 12
```

```r
# Show column names
cat("\nField data columns:\n")
```

```
##
## Field data columns:
```

```r
print(names(df_field))
```

```
##  [1] "timestamp"                 "user"
##  [3] "ai"                        "conversation_id"
##  [5] "respondent_id"             "interaction_count"
##  [7] "llm_type"                  "turn"
##  [9] "starter"                   "user_embedding"
## [11] "ai_embedding"              "pct_turn"
## [13] "user_sentiment_score"      "ai_sentiment_score"
## [15] "user_sentiment_projection" "ai_sentiment_projection"
```

```r
cat("\nSimulated data columns:\n")
```

```
##
## Simulated data columns:
```

```r
print(names(df_simulated))
```

```
##  [1] "turn"                 "user"                 "ai"
##  [4] "client_id"            "workshop_id"          "conversation_id"
```

```
##  [7] "timestamp"            "user_embedding"       "ai_embedding"
## [10] "pct_turn"             "user_sentiment_score" "ai_sentiment_score"
```

1.2 1.2 Data Preprocessing

```r
# Preprocess field data
df_field <- df_field %>%
  rename(
    ai_valence = ai_sentiment_projection,
    user_valence = user_sentiment_projection
  ) %>%
  mutate(
    # Token counts
    user_token_count = stri_count_words(user),
    ai_token_count = stri_count_words(ai),
    token_ratio = if_else(ai_token_count == 0, NA_real_,
                          user_token_count / ai_token_count),

    # Time parsing
    timestamp = ymd_hms(timestamp),

    # Data source label
    data_source = "Field"
  ) %>%
  # Ensure conversation_id is a character/factor
  mutate(conversation_id = as.character(conversation_id)) %>%
  filter(turn > 1)


# Preprocess simulated data
df_simulated <- df_simulated %>%
  rename(
    ai_valence = ai_sentiment_score,
    user_valence = user_sentiment_score
  ) %>%
  mutate(
    # Token counts
    user_token_count = stri_count_words(user),
    ai_token_count = stri_count_words(ai),
    token_ratio = if_else(ai_token_count == 0, NA_real_,
                          user_token_count / ai_token_count),

    # Time parsing (if available)
    timestamp = if_else(!is.na(timestamp), ymd_hms(timestamp), as.POSIXct(NA)),

    # Data source label
    data_source = "Simulated"
  ) %>%
  mutate(conversation_id = as.character(conversation_id))

# Summary statistics
cat("\n=== Field Data Summary ===\n")
```

```
##
```

```
## === Field Data Summary ===
```

```r
df_field %>%
  summarise(
    n_conversations = n_distinct(conversation_id),
    n_turns = n(),
    mean_turns_per_story = n() / n_distinct(conversation_id),
    mean_user_valence = mean(user_valence, na.rm = TRUE),
    mean_ai_valence = mean(ai_valence, na.rm = TRUE)
  ) %>%
  print()
```

```
## # A tibble: 1 x 5
##   n_conversations n_turns mean_turns_per_story mean_user_valence mean_ai_valence
##             <int>   <int>                <dbl>             <dbl>           <dbl>
## 1              82     738                    9           -0.0890        -0.00147
```

```r
cat("\n=== Simulated Data Summary ===\n")
```

```
##
## === Simulated Data Summary ===
```

```r
df_simulated %>%
  summarise(
    n_conversations = n_distinct(conversation_id),
    n_turns = n(),
    mean_turns_per_story = n() / n_distinct(conversation_id),
    mean_user_valence = mean(user_valence, na.rm = TRUE),
    mean_ai_valence = mean(ai_valence, na.rm = TRUE)
  ) %>%
  print()
```

```
## # A tibble: 1 x 5
##   n_conversations n_turns mean_turns_per_story mean_user_valence mean_ai_valence
##             <int>   <int>                <dbl>             <dbl>           <dbl>
## 1               6      30                    5            -0.407          -0.296
```

1.3 1.3 Compute Lagged Variables and Cosine

```r
# Add lagged AI valence for field data
# Note: lag() within group_by ensures no cross-story contamination
# Turn 1 of each story will have NA for lagged values (correct behavior)

df_field <- df_field %>%
  group_by(conversation_id) %>%
  arrange(turn, .by_group = TRUE) %>%
  mutate(
    ai_valence_lag1 = lag(ai_valence, n = 1),
    user_valence_lag1 = lag(user_valence, n = 1),
  ) %>%
  ungroup()


cosine_sim <- function(x, y) {
  if (is.null(x) || is.null(y) || any(is.na(x)) || any(is.na(y))) {
    return(NA_real_)
```

```r
  }
  return(sum(x * y) / (sqrt(sum(x^2)) * sqrt(sum(y^2))))
}

df_field <- df_field %>%
  group_by(conversation_id) %>%
  arrange(turn, .by_group = TRUE) %>%
  mutate(
    ai_embedding_prev = lag(ai_embedding)
  ) %>%
  rowwise() %>%
  mutate(
    cosine_sim_same = cosine_sim(user_embedding, ai_embedding),
    cosine_sim_prev = cosine_sim(user_embedding, ai_embedding_prev)
  ) %>%
  ungroup()

# Add lagged AI valence for simulated data
df_simulated <- df_simulated %>%
  group_by(conversation_id) %>%
  arrange(turn, .by_group = TRUE) %>%
  mutate(
    ai_valence_lag1 = lag(ai_valence, n = 1),
    user_valence_lag1 = lag(user_valence, n = 1)
  ) %>%
  ungroup()

# Verify no cross-story contamination
cat("=== Lag Variable Verification ===\n")
```

```
## === Lag Variable Verification ===
```

```r
cat("Field data - Turn 1 with non-NA lag (should be 0):",
    sum(df_field$turn == 1 & !is.na(df_field$ai_valence_lag1)), "\n")
```

```
## Field data - Turn 1 with non-NA lag (should be 0): 0
```

```r
cat("Simulated data - Turn 1 with non-NA lag (should be 0):",
    sum(df_simulated$turn == 1 & !is.na(df_simulated$ai_valence_lag1)), "\n")
```

```
## Simulated data - Turn 1 with non-NA lag (should be 0): 0
```

2 2. Descriptive Statistics

2.1 2.1 Token Count Statistics

```r
# Field data token statistics
token_stats_field <- df_field %>%
  summarise(
    mean_user_tokens = mean(user_token_count, na.rm = TRUE),
    sd_user_tokens = sd(user_token_count, na.rm = TRUE),
    mean_ai_tokens = mean(ai_token_count, na.rm = TRUE),
    sd_ai_tokens = sd(ai_token_count, na.rm = TRUE),
    mean_token_ratio = mean(token_ratio, na.rm = TRUE)
```

```r
  )

cat("=== Field Data Token Statistics ===\n")
```

```
## === Field Data Token Statistics ===
```

```r
print(token_stats_field)
```

```
## # A tibble: 1 x 5
##   mean_user_tokens sd_user_tokens mean_ai_tokens sd_ai_tokens mean_token_ratio
##              <dbl>          <dbl>          <dbl>        <dbl>            <dbl>
## 1             25.0           5.41           28.5         7.68            0.920
```

```r
# Simulated data token statistics
token_stats_sim <- df_simulated %>%
  summarise(
    mean_user_tokens = mean(user_token_count, na.rm = TRUE),
    sd_user_tokens = sd(user_token_count, na.rm = TRUE),
    mean_ai_tokens = mean(ai_token_count, na.rm = TRUE),
    sd_ai_tokens = sd(ai_token_count, na.rm = TRUE),
    mean_token_ratio = mean(token_ratio, na.rm = TRUE)
  )

cat("\n=== Simulated Data Token Statistics ===\n")
```

```
##
## === Simulated Data Token Statistics ===
```

```r
print(token_stats_sim)
```

```
## # A tibble: 1 x 5
##   mean_user_tokens sd_user_tokens mean_ai_tokens sd_ai_tokens mean_token_ratio
##              <dbl>          <dbl>          <dbl>        <dbl>            <dbl>
## 1             26.0           2.17           41.1         3.73            0.636
```

2.2 2.2 Valence Distribution Statistics

```r
# Combined statistics
valence_stats <- bind_rows(
  df_field %>%
    pivot_longer(cols = c(user_valence, ai_valence),
                 names_to = "agent", values_to = "valence") %>%
    mutate(agent = str_remove(agent, "_valence")) %>%
    group_by(data_source, agent) %>%
    summarise(
      mean_valence = mean(valence, na.rm = TRUE),
      sd_valence = sd(valence, na.rm = TRUE),
      median_valence = median(valence, na.rm = TRUE),
      .groups = "drop"
    ),

  df_simulated %>%
    pivot_longer(cols = c(user_valence, ai_valence),
                 names_to = "agent", values_to = "valence") %>%
    mutate(agent = str_remove(agent, "_valence")) %>%
    group_by(data_source, agent) %>%
```

```r
    summarise(
      mean_valence = mean(valence, na.rm = TRUE),
      sd_valence = sd(valence, na.rm = TRUE),
      median_valence = median(valence, na.rm = TRUE),
      .groups = "drop"
    )
)

print(valence_stats)
```

```
## # A tibble: 4 x 5
##   data_source agent mean_valence sd_valence median_valence
##   <chr>       <chr>        <dbl>      <dbl>          <dbl>
## 1 Field       ai        -0.00147      0.402        -0.0212
## 2 Field       user      -0.0890       0.395        -0.108
## 3 Simulated   ai        -0.296        0.681        -0.487
## 4 Simulated   user      -0.407        0.529        -0.504
```

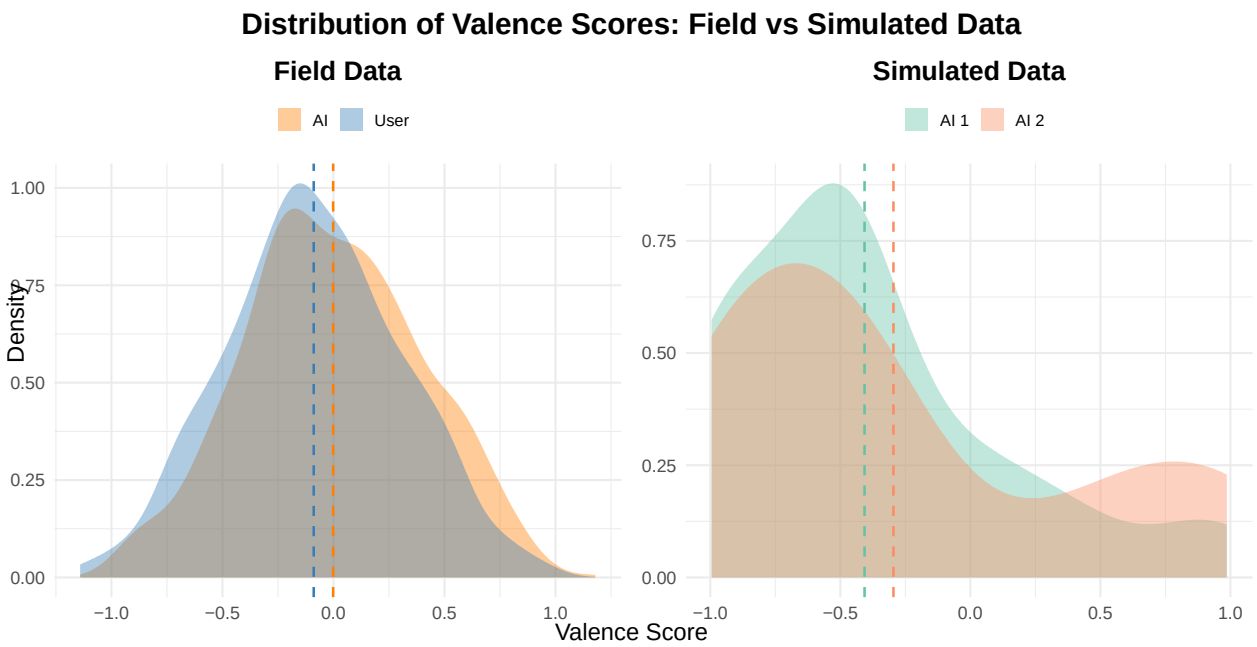
```r
# Save to CSV
write_csv(valence_stats, file.path(TABLES_DIR, "valence_descriptive_stats.csv"))
```

3 3. Valence Distribution Analysis

3.1 3.1 Combined Valence Distribution Plot

```r
# Prepare data for plotting
df_field_long <- df_field %>%
  pivot_longer(cols = c(user_valence, ai_valence),
               names_to = "agent", values_to = "valence") %>%
  mutate(agent = recode(agent,
                        user_valence = "User",
                        ai_valence = "AI"))

df_sim_long <- df_simulated %>%
  pivot_longer(cols = c(user_valence, ai_valence),
               names_to = "agent", values_to = "valence") %>%
  mutate(agent = recode(agent,
                        user_valence = "AI 1",
                        ai_valence = "AI 2"))


# Calculate means for vertical lines
means_field <- df_field_long %>%
  group_by(agent) %>%
  summarise(mean_val = mean(valence, na.rm = TRUE), .groups = "drop")

means_sim <- df_sim_long %>%
  group_by(agent) %>%
  summarise(mean_val = mean(valence, na.rm = TRUE), .groups = "drop")

# Field data plot
p_field <- ggplot(df_field_long, aes(x = valence, fill = agent)) +
  geom_density(alpha = 0.4, color = NA) +
```

```r
  geom_vline(data = means_field,
             aes(xintercept = mean_val, color = agent),
             linetype = "dashed", linewidth = 0.8, show.legend = FALSE) +
  scale_fill_manual(values = c("User" = "#377EB8", "AI" = "#FF7F00"), name = "") +
  scale_color_manual(values = c("User" = "#377EB8", "AI" = "#FF7F00")) +
  labs(title = "Field Data", x = NULL, y = NULL) +
  theme(
    legend.position = "top",
    plot.title = element_text(hjust = 0.5, size = 18, face = "bold"),
    axis.text = element_text(size = 12)
  )

# Simulated data plot
p_sim <- ggplot(df_sim_long, aes(x = valence, fill = agent)) +
  geom_density(alpha = 0.4, color = NA) +
  geom_vline(data = means_sim,
             aes(xintercept = mean_val, color = agent),
             linetype = "dashed", linewidth = 0.8, show.legend = FALSE) +
  scale_fill_brewer(palette = "Set2", name = "") +
  scale_color_brewer(palette = "Set2") +
  labs(title = "Simulated Data", x = NULL, y = NULL) +
  theme(
    legend.position = "top",
    plot.title = element_text(hjust = 0.5, size = 18, face = "bold"),
    axis.text = element_text(size = 12)
  )

# Combine plots
combined_plot <- (p_field | p_sim) +
  plot_annotation(
    title = "Distribution of Valence Scores: Field vs Simulated Data",
    theme = theme(plot.title = element_text(hjust = 0.5, size = 20, face = "bold"))
  )

# Add axis labels using cowplot
final_valence_dist <- ggdraw() +
  draw_plot(combined_plot, 0, 0, 1, 1) +
  draw_label("Valence Score", x = 0.5, y = 0.02, size = 16) +
  draw_label("Density", x = 0.02, y = 0.5, angle = 90, size = 16)

print(final_valence_dist)
```




```
ggsave(file.path(GRAPHS_DIR, "valence_distribution_comparison.png"),
       final_valence_dist, width = 12, height = 6, dpi = 300)
```

4 4. Valence Alignment Analysis

4.1 4.1 Per-Story Correlation (Same Turn)

```
# Field data: same-turn correlation
cor_field_same <- df_field %>%
  group_by(conversation_id) %>%
  filter(n() >= 3) %>%  # Require at least 3 turns
  summarise(
    rho = cor(user_valence, ai_valence, use = "complete.obs"),
    n_turns = n(),
    data_source = "Field",
    .groups = "drop"
  )

# Simulated data: same-turn correlation
cor_sim_same <- df_simulated %>%
  group_by(conversation_id) %>%
  filter(n() >= 3) %>%
  summarise(
    rho = cor(user_valence, ai_valence, use = "complete.obs"),
    n_turns = n(),
    data_source = "Simulated",
    .groups = "drop"
  )

# Combine
cor_same_combined <- bind_rows(cor_field_same, cor_sim_same) %>%
  mutate(alignment_type = "Same Turn")
```

```r
# Summary
cat("=== Same-Turn Alignment Summary ===\n")
```

```
## === Same-Turn Alignment Summary ===
```

```r
cor_same_combined %>%
  group_by(data_source) %>%
  summarise(
    mean_rho = mean(rho, na.rm = TRUE),
    sd_rho = sd(rho, na.rm = TRUE),
    median_rho = median(rho, na.rm = TRUE),
    n_stories = n()
  ) %>%
  print()
```

```
## # A tibble: 2 x 5
##   data_source mean_rho sd_rho median_rho n_stories
##   <chr>          <dbl>  <dbl>      <dbl>     <int>
## 1 Field          0.230  0.353      0.250        82
## 2 Simulated     -0.246  0.440     -0.314         6
```

4.2 4.2 Per-Story Correlation (Previous Turn)

```r
# Field data: previous-turn correlation (User response to previous AI)
cor_field_prev <- df_field %>%
  group_by(conversation_id) %>%
  filter(!is.na(ai_valence_lag1)) %>%
  filter(n() >= 3) %>%
  summarise(
    rho = cor(user_valence, ai_valence_lag1, use = "complete.obs"),
    n_turns = n(),
    data_source = "Field",
    .groups = "drop"
  )

# Simulated data: previous-turn correlation
cor_sim_prev <- df_simulated %>%
  group_by(conversation_id) %>%
  filter(!is.na(ai_valence_lag1)) %>%
  filter(n() >= 3) %>%
  summarise(
    rho = cor(user_valence, ai_valence_lag1, use = "complete.obs"),
    n_turns = n(),
    data_source = "Simulated",
    .groups = "drop"
  )

# Combine
cor_prev_combined <- bind_rows(cor_field_prev, cor_sim_prev) %>%
  mutate(alignment_type = "Previous Turn")

# Summary
cat("=== Previous-Turn Alignment Summary ===\n")
```

```
## === Previous-Turn Alignment Summary ===
```

```r
cor_prev_combined %>%
  group_by(data_source) %>%
  summarise(
    mean_rho = mean(rho, na.rm = TRUE),
    sd_rho = sd(rho, na.rm = TRUE),
    median_rho = median(rho, na.rm = TRUE),
    n_stories = n()
  ) %>%
  print()
```

```
## # A tibble: 2 x 5
##   data_source mean_rho sd_rho median_rho n_stories
##   <chr>          <dbl>  <dbl>      <dbl>     <int>
## 1 Field         0.0861  0.346      0.129        82
## 2 Simulated     0.533   0.224      0.546         6
```

4.3 4.3.1 All turns story correlation (same turn)

```r
#Violates assumptions since turns are not independent (nested in stories)
cor_field_same_all_turns <- df_field %>%
  summarise(
    rho = cor(user_valence, ai_valence, use = "complete.obs"),
    n_turns = n(),
    data_source = "Field"
  )

cor_sim_same_all_turns <- df_simulated %>%
  summarise(
    rho = cor(user_valence, ai_valence, use = "complete.obs"),
    n_turns = n(),
    data_source = "Simulated"
  )

cor_same_combined_all_turns <- bind_rows(cor_field_same_all_turns, cor_sim_same_all_turns) %>%
  mutate(alignment_tupe = "Same Turn All Turns")

cat("=== Same-Turn All Turns Alignment Summary ===\n")
```

```
## === Same-Turn All Turns Alignment Summary ===
```

```r
cor_same_combined_all_turns %>%
  group_by(data_source) %>%
  summarise(
    mean_rho = mean(rho, na.rm = TRUE),
    sd_rho = sd(rho, na.rm = TRUE),
    median_rho = median(rho, na.rm = TRUE),
    n_stories = n()
  ) %>%
  print()
```

```
## # A tibble: 2 x 5
##   data_source mean_rho sd_rho median_rho n_stories
##   <chr>          <dbl>  <dbl>      <dbl>     <int>
```

```
## 1 Field          0.428     NA      0.428         1
## 2 Simulated     -0.110     NA     -0.110         1
```

4.4 4.3.2 All turns story correlation (Previous Turn)

```r
# Violates independence assumption (turns nested in stories)
# Note: use="complete.obs" automatically excludes turn 1 (where ai_valence_lag1 is NA)
# This ensures user_t1 is never paired with ai from previous story

# Verify exclusion of turn 1
cat("=== Previous-Turn Aggregate Correlation ===\n")
```

```
## === Previous-Turn Aggregate Correlation ===
```

```r
cat("Field: Turn 1 observations (excluded):", sum(df_field$turn == 1), "\n")
```

```
## Field: Turn 1 observations (excluded): 0
```

```r
cat("Field: Valid lag observations:", sum(!is.na(df_field$ai_valence_lag1)), "\n")
```

```
## Field: Valid lag observations: 656
```

```r
cat("Simulated: Turn 1 observations (excluded):", sum(df_simulated$turn == 1), "\n")
```

```
## Simulated: Turn 1 observations (excluded): 6
```

```r
cat("Simulated: Valid lag observations:", sum(!is.na(df_simulated$ai_valence_lag1)), "\n\n")
```

```
## Simulated: Valid lag observations: 24
```

```r
cor_field_same_all_turns_prev <- df_field %>%
  summarise(
    rho = cor(user_valence, ai_valence_lag1, use = "complete.obs"),
    n_turns = n(),
    n_valid = sum(!is.na(ai_valence_lag1)),
    data_source = "Field"
  )

cor_sim_same_all_turns_prev <- df_simulated %>%
  summarise(
    rho = cor(user_valence, ai_valence_lag1, use = "complete.obs"),
    n_turns = n(),
    n_valid = sum(!is.na(ai_valence_lag1)),
    data_source = "Simulated"
  )

cor_same_combined_all_turns_prev <- bind_rows(cor_field_same_all_turns_prev, cor_sim_same_all_turns_prev
  mutate(alignment_type = "Previous Turn All Turns")

cat("=== Previous-Turn All Turns Alignment Summary ===\n")
```

```
## === Previous-Turn All Turns Alignment Summary ===
```

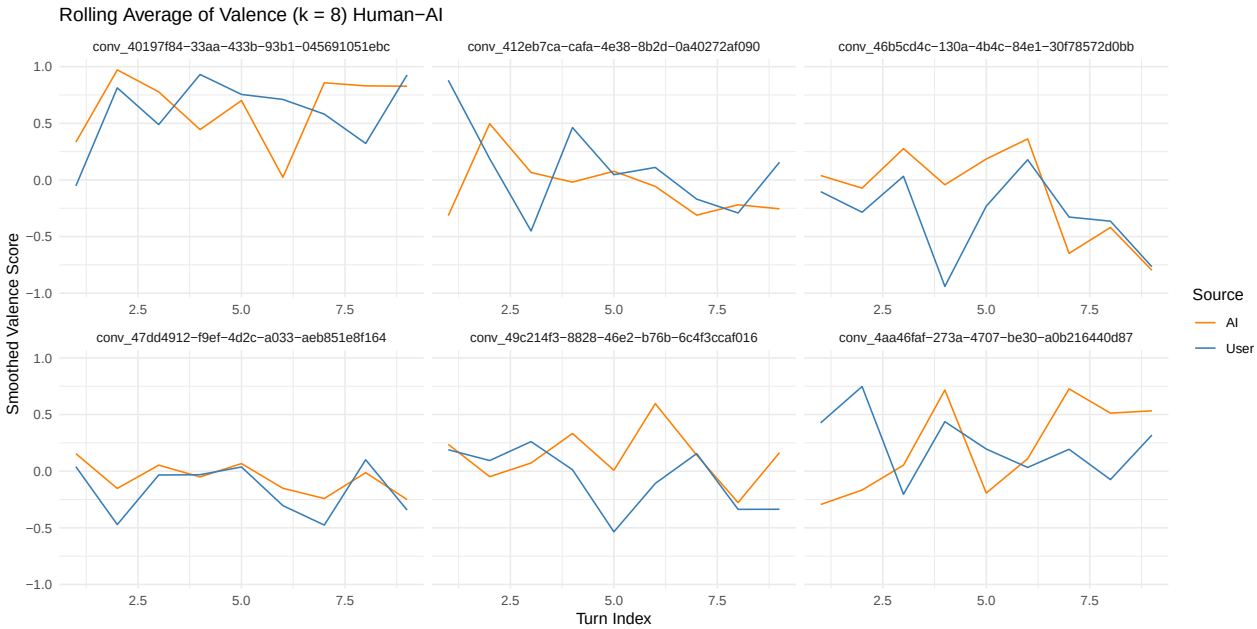
```r
cor_same_combined_all_turns_prev %>%
  group_by(data_source) %>%
  summarise(
    mean_rho = mean(rho, na.rm = TRUE),
    sd_rho = sd(rho, na.rm = TRUE),
```

```r
    median_rho = median(rho, na.rm = TRUE),
    n_total = sum(n_turns),
    n_valid = sum(n_valid)
  ) %>%
  print()
```

```
## # A tibble: 2 x 6
##   data_source mean_rho sd_rho median_rho n_total n_valid
##   <chr>          <dbl>  <dbl>      <dbl>   <int>   <int>
## 1 Field          0.355     NA      0.355     738     656
## 2 Simulated      0.425     NA      0.425      30      24
```

4.5 4.3.3 Probe Sentiment Narrative Development

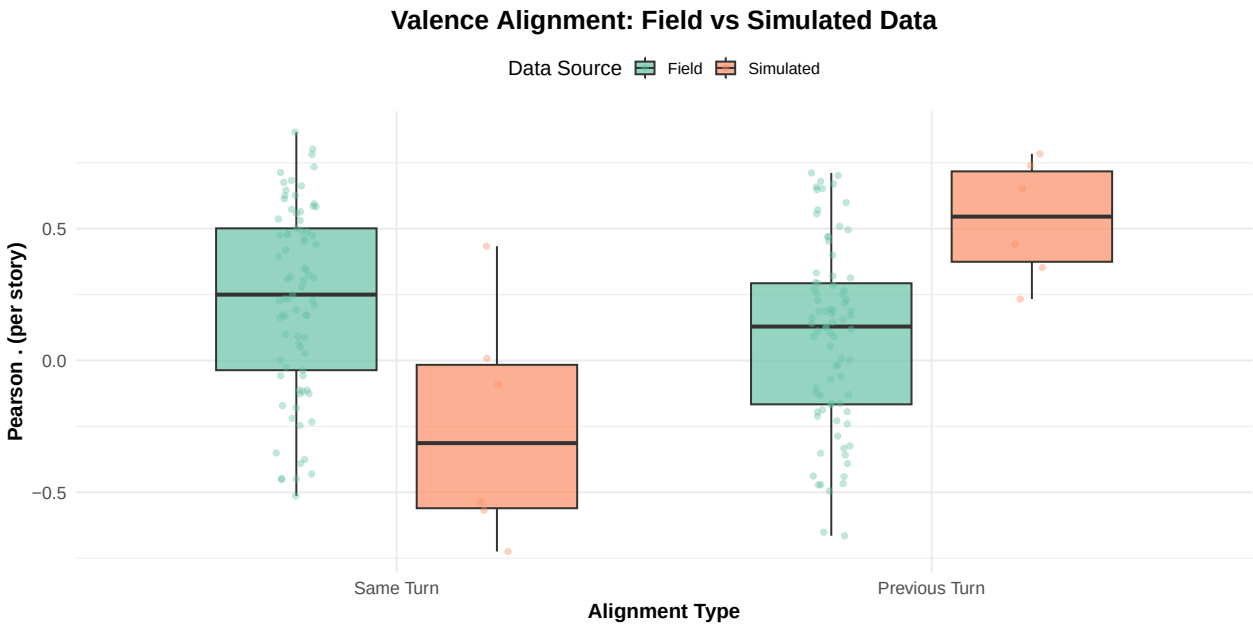
```r
df_field %>%
  mutate(time = ymd_hms(timestamp)) %>%

  group_by(conversation_id) %>%
  arrange(time, .by_group = TRUE) %>%
  mutate(turn_index = row_number()) %>%
  ungroup() %>%
  filter(conversation_id %in% unique(conversation_id)[20:25]) %>%
  pivot_longer(cols = c(user_valence, ai_valence),
               names_to = "source", values_to = "valence") %>%
  mutate(
    source = recode(source, "user_valence" = "User", "ai_valence" = "AI"),
    valence_smoothed = rollmean(valence, k = 1, fill = NA, align = "center")
  ) %>%
  ggplot(aes(x = turn_index, y = valence_smoothed, color = source)) +
  geom_line() +
  scale_color_manual(values = c("User" = "#377EB8", "AI" = "#FF7F00")) +
  facet_wrap(~conversation_id, scales = "free_x") +
  labs(
    title = "Rolling Average of Valence (k = 8) Human-AI",
    x = "Turn Index",
    y = "Smoothed Valence Score",
    color = "Source"
  ) +
  theme_minimal()
```



4.6 4.4.1 Alignment Comparison Plot

```
# Combine all correlations
cor_all <- bind_rows(cor_same_combined, cor_prev_combined) %>%
  mutate(
    alignment_type = factor(alignment_type,
                            levels = c("Same Turn", "Previous Turn"))
  )

# Boxplot with jittered points
p_alignment <- ggplot(cor_all, aes(x = alignment_type, y = rho, fill = data_source)) +
  geom_boxplot(width = 0.6, outlier.shape = NA, alpha = 0.7,
               position = position_dodge(width = 0.75)) +
  geom_jitter(aes(color = data_source),
              position = position_jitterdodge(
                dodge.width = 0.75,
                jitter.width = 0.15
              ),
              size = 1.5, alpha = 0.4) +
  scale_fill_brewer(palette = "Set2", name = "Data Source") +
  scale_color_brewer(palette = "Set2", name = "Data Source") +
  labs(
    title = "Valence Alignment: Field vs Simulated Data",
    x = "Alignment Type",
    y = "Pearson   (per story)"
  ) +
  theme(
    legend.position = "top",
    plot.title = element_text(hjust = 0.5, face = "bold", size = 18),
    axis.title = element_text(size = 14, face = "bold"),
    axis.text = element_text(size = 12)
  )
```

```r
print(p_alignment)
```

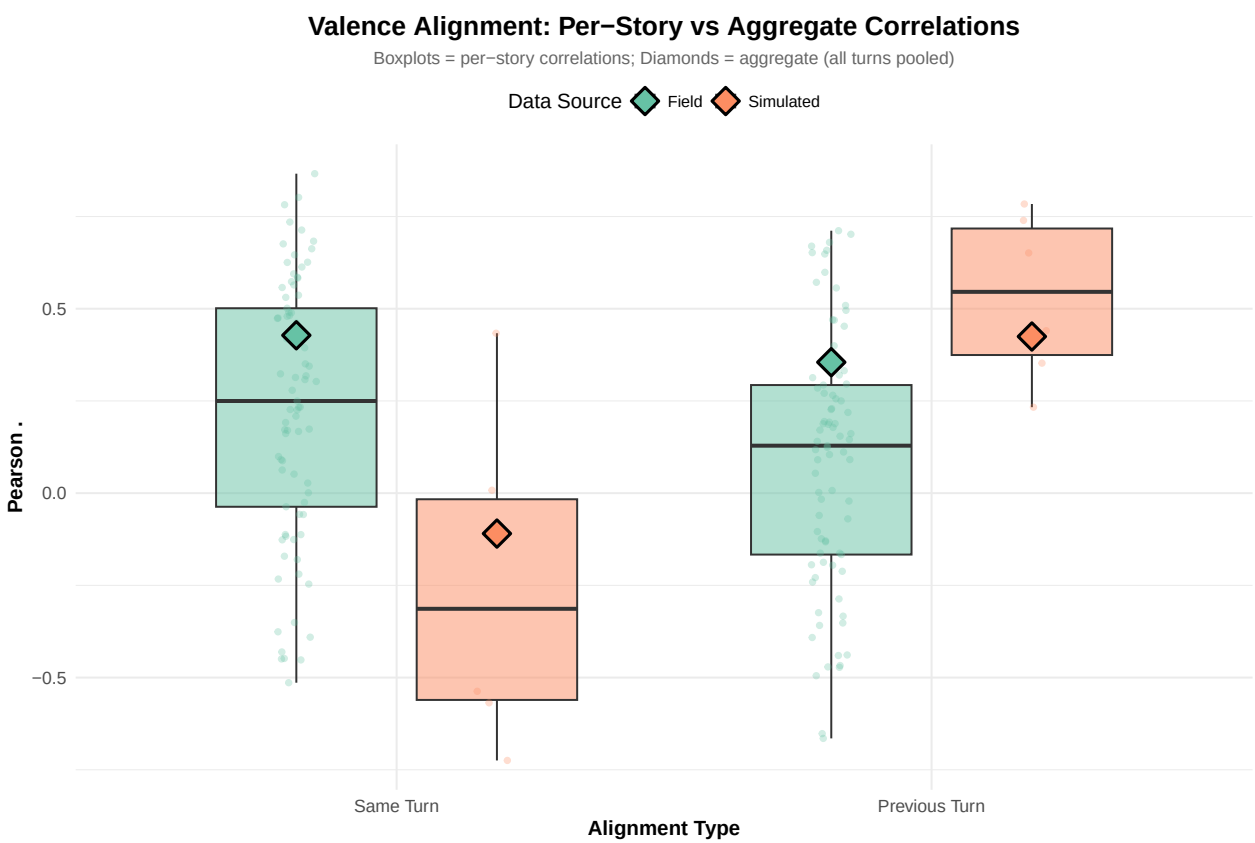



```r
ggsave(file.path(GRAPHS_DIR, "valence_alignment_comparison.png"),
       p_alignment, width = 12, height = 6, dpi = 300)
```

4.7 4.4.2 Comparison: Per-Story vs Aggregate Correlations

```r
# Prepare aggregate correlation data for overlay
agg_same <- cor_same_combined_all_turns %>%
  mutate(alignment_type = "Same Turn")

agg_prev <- cor_same_combined_all_turns_prev %>%
  mutate(alignment_type = "Previous Turn") %>%
  select(data_source, rho, alignment_type)

agg_data <- bind_rows(agg_same, agg_prev) %>%
  mutate(
    alignment_type = factor(alignment_type,
                            levels = c("Same Turn", "Previous Turn"))
  )

# Create plot with per-story boxplots and aggregate overlay
p_comparison <- ggplot(cor_all, aes(x = alignment_type, y = rho, fill = data_source)) +
  # Per-story boxplots
  geom_boxplot(width = 0.6, outlier.shape = NA, alpha = 0.5,
               position = position_dodge(width = 0.75)) +
  # Per-story jittered points
  geom_jitter(aes(color = data_source),
              position = position_jitterdodge(
                dodge.width = 0.75,
                jitter.width = 0.15
              ),
              size = 1.5, alpha = 0.3) +
```

```r
  # Aggregate correlations as large points
  geom_point(data = agg_data,
             aes(x = alignment_type, y = rho, fill = data_source),
             position = position_dodge(width = 0.75),
             size = 6, shape = 23, color = "black", stroke = 1.5) +
  scale_fill_brewer(palette = "Set2", name = "Data Source") +
  scale_color_brewer(palette = "Set2", name = "Data Source") +
  labs(
    title = "Valence Alignment: Per-Story vs Aggregate Correlations",
    subtitle = "Boxplots = per-story correlations; Diamonds = aggregate (all turns pooled)",
    x = "Alignment Type",
    y = "Pearson  "
  ) +
  theme(
    legend.position = "top",
    plot.title = element_text(hjust = 0.5, face = "bold", size = 18),
    plot.subtitle = element_text(hjust = 0.5, size = 12, color = "grey40"),
    axis.title = element_text(size = 14, face = "bold"),
    axis.text = element_text(size = 12)
  )

print(p_comparison)
```




```r
ggsave(file.path(GRAPHS_DIR, "valence_alignment_comparison_with_aggregate.png"),
       p_comparison, width = 12, height = 8, dpi = 300)

# Print summary comparison
```

```r
cat("\n=== Per-Story vs Aggregate Correlation Comparison ===\n")
```

```
##
## === Per-Story vs Aggregate Correlation Comparison ===
```

```r
comparison_summary <- bind_rows(
  cor_all %>%
    group_by(data_source, alignment_type) %>%
    summarise(
      mean_rho = mean(rho, na.rm = TRUE),
      method = "Per-Story Mean",
      .groups = "drop"
    ),
  agg_data %>%
    mutate(mean_rho = rho, method = "Aggregate All Turns") %>%
    select(data_source, alignment_type, mean_rho, method)
) %>%
  pivot_wider(names_from = method, values_from = mean_rho)

print(comparison_summary)
```

```
## # A tibble: 4 x 4
##   data_source alignment_type `Per-Story Mean` `Aggregate All Turns`
##   <chr>       <fct>                    <dbl>                 <dbl>
## 1 Field       Same Turn                0.230                 0.428
## 2 Field       Previous Turn            0.0861                0.355
## 3 Simulated   Same Turn               -0.246                -0.110
## 4 Simulated   Previous Turn            0.533                 0.425
```

4.8 4.5 Mixed Effects Model: Random Intercepts per Story

```r
# Same-turn alignment with random intercepts per story
model_same <- lmer(ai_valence ~ user_valence + (1 | conversation_id), data = df_field)
cat("=== Mixed Effects Model: Same-Turn Alignment (Field Data) ===\n")
```

```
## === Mixed Effects Model: Same-Turn Alignment (Field Data) ===
```

```r
print(summary(model_same)$coefficients)
```

```
##                Estimate Std. Error        df  t value     Pr(>|t|)
## (Intercept)  0.02455583 0.02423491  77.19148 1.013242 3.141101e-01
## user_valence 0.29259084 0.03413747 735.85476 8.570960 6.007866e-17
```

```r
# Previous-turn alignment with random intercepts per story
model_prev <- lmer(user_valence ~ ai_valence_lag1 + (1 | conversation_id), data = df_field)
cat("\n=== Mixed Effects Model: Previous-Turn Alignment (Field Data) ===\n")
```

```
##
## === Mixed Effects Model: Previous-Turn Alignment (Field Data) ===
```

```r
print(summary(model_prev)$coefficients)
```

```
##                    Estimate Std. Error        df   t value     Pr(>|t|)
## (Intercept)     -0.09952863 0.02232139  69.47634 -4.458890 3.089381e-05
## ai_valence_lag1  0.22381700 0.03950335 601.46520  5.665772 2.271078e-08
```

```r
# We assume df_field already has:


# 1) Build long-format dataset with both directions in a common structure

# Same-turn: User -> AI
df_same_dir <- df_field %>%
  transmute(
    conversation_id,
    turn,
    alignment_direction = "User_to_AI",
    predictor_valence = user_valence,
    response_valence = ai_valence
  )

# Previous-turn: AI -> User (response to previous AI valence)
df_prev_dir <- df_field %>%
  filter(!is.na(ai_valence_lag1)) %>%   # ensure valid lag values
  transmute(
    conversation_id,
    turn,
    alignment_direction = "AI_to_User",
    predictor_valence = ai_valence_lag1,
    response_valence = user_valence
  )

# Combine into one long dataset
df_alignment_long <- bind_rows(df_same_dir, df_prev_dir) %>%
  mutate(
    alignment_direction = factor(
      alignment_direction,
      levels = c("User_to_AI", "AI_to_User")
    )
  )

cat("=== Directional Alignment Long-Format Data ===\n")
```

```
## === Directional Alignment Long-Format Data ===
```

```r
df_alignment_long %>%
  group_by(alignment_direction) %>%
  summarise(
    n_rows = n(),
    mean_predictor = mean(predictor_valence, na.rm = TRUE),
    mean_response  = mean(response_valence,  na.rm = TRUE),
    n_conversations = n_distinct(conversation_id),
    .groups = "drop"
  ) %>%
  print()
```

```
## # A tibble: 2 x 5
##   alignment_direction n_rows mean_predictor mean_response n_conversations
##   <fct>               <int>          <dbl>         <dbl>           <int>
## 1 User_to_AI            738       -0.0890       -0.00147              82
```

```
## 2 AI_to_User             656       -0.00302      -0.100                 82
```

```r
# 2) Mixed-effects model comparing slopes across directions
# Reference level: User_to_AI
# Key term: predictor_valence:alignment_directionAI_to_User

model_direction <- lmer(
  response_valence ~ predictor_valence * alignment_direction +
    (1 | conversation_id),
  data = df_alignment_long
)

cat("=== Mixed Effects Model: Directional Alignment (User -> AI vs AI -> User) ===\n")
```

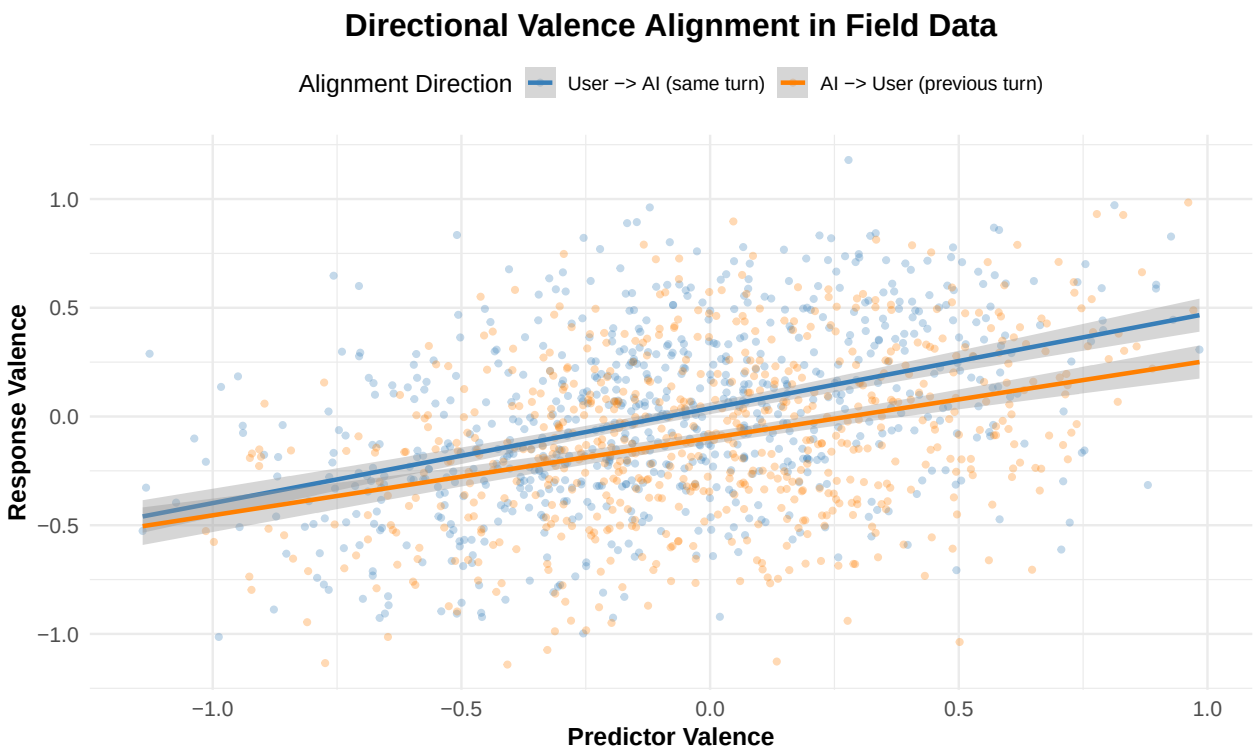
```
## === Mixed Effects Model: Directional Alignment (User -> AI vs AI -> User) ===
```

```r
direction_summary <- summary(model_direction)
print(direction_summary$coefficients)
```

```
##                                                   Estimate Std. Error
## (Intercept)                                     0.01735372 0.02415194
## predictor_valence                               0.21162437 0.03351051
## alignment_directionAI_to_User                  -0.11731579 0.01784963
## predictor_valence:alignment_directionAI_to_User -0.13135955 0.04598847
##                                                        df    t value
## (Intercept)                                      92.66411  0.7185227
## predictor_valence                              1381.21278  6.3151632
## alignment_directionAI_to_User                  1299.34719 -6.5724500
## predictor_valence:alignment_directionAI_to_User 1321.93299 -2.8563584
##                                                    Pr(>|t|)
## (Intercept)                                    4.742429e-01
## predictor_valence                              3.630376e-10
## alignment_directionAI_to_User                  7.147176e-11
## predictor_valence:alignment_directionAI_to_User 4.352351e-03
```

```r
# 3) Visualise directional alignment with separate regression lines

p_direction_alignment <- df_alignment_long %>%
  ggplot(aes(
    x = predictor_valence,
    y = response_valence,
    color = alignment_direction
  )) +
  geom_point(alpha = 0.3, size = 1.3) +
  geom_smooth(
    method = "lm",
    se = TRUE,
    fullrange = TRUE,
    size = 1.2
  ) +
  scale_color_manual(
    values = c("User_to_AI" = "#377EB8", "AI_to_User" = "#FF7F00"),
    name = "Alignment Direction",
    labels = c("User → AI (same turn)", "AI → User (previous turn)")
  ) +
  labs(
```

```
    title = "Directional Valence Alignment in Field Data",
    x = "Predictor Valence",
    y = "Response Valence"
  ) +
  theme_minimal(base_size = 14) +
  theme(
    legend.position = "top",
    plot.title = element_text(hjust = 0.5, face = "bold", size = 18),
    axis.title = element_text(size = 14, face = "bold"),
    axis.text = element_text(size = 12)
  )

print(p_direction_alignment)
```




```
ggsave(
  file.path(GRAPHS_DIR, "directional_valence_alignment_field.png"),
  p_direction_alignment,
  width = 10,
  height = 6,
  dpi = 300
)
```

4.9 4.6 Statistical Tests: Alignment Differences

```
# Test 1: Same-turn alignment difference
t_test_same <- t.test(rho ~ data_source, data = cor_same_combined)
cat("=== T-test: Same-Turn Alignment ===\n")
```

```
## === T-test: Same-Turn Alignment ===
```

```r
print(t_test_same)
```

```
##
##  Welch Two Sample t-test
##
## data:  rho by data_source
## t = 2.5925, df = 5.4868, p-value = 0.0446
## alternative hypothesis: true difference in means between group Field and group Simulated is not equal
## 95 percent confidence interval:
##  0.01636547 0.93717286
## sample estimates:
##     mean in group Field mean in group Simulated
##               0.2303227              -0.2464464
```

```r
# Test 2: Previous-turn alignment difference
t_test_prev <- t.test(rho ~ data_source, data = cor_prev_combined)
cat("\n=== T-test: Previous-Turn Alignment ===\n")
```

```
##
## === T-test: Previous-Turn Alignment ===
```

```r
print(t_test_prev)
```

```
##
##  Welch Two Sample t-test
##
## data:  rho by data_source
## t = -4.5099, df = 6.9129, p-value = 0.002853
## alternative hypothesis: true difference in means between group Field and group Simulated is not equal
## 95 percent confidence interval:
##  -0.6824199 -0.2121730
## sample estimates:
##     mean in group Field mean in group Simulated
##               0.0861384               0.5334349
```

```r
# Test 3: One-sample t-tests against zero (is alignment significant?)
cat("\n=== One-Sample Tests Against Zero ===\n")
```

```
##
## === One-Sample Tests Against Zero ===
```

```r
cor_all %>%
  group_by(data_source, alignment_type) %>%
  summarise(
    t_stat = t.test(rho, mu = 0)$statistic,
    p_value = t.test(rho, mu = 0)$p.value,
    mean_rho = mean(rho, na.rm = TRUE),
    n = n(),
    .groups = "drop"
  ) %>%
  print()
```

```
## # A tibble: 4 x 6
##   data_source alignment_type t_stat      p_value mean_rho     n
##   <chr>       <fct>           <dbl>        <dbl>    <dbl> <int>
## 1 Field       Same Turn        5.87 0.0000000931   0.230     82
## 2 Field       Previous Turn    2.24 0.0279         0.0861    82
```

```
## 3 Simulated   Same Turn       -1.37 0.229         -0.246      6
## 4 Simulated   Previous Turn    5.84 0.00209        0.533      6
```

```r
# Save results
alignment_test_results <- tibble(
  test = c("Same-turn: Field vs Simulated", "Previous-turn: Field vs Simulated"),
  t_statistic = c(t_test_same$statistic, t_test_prev$statistic),
  p_value = c(t_test_same$p.value, t_test_prev$p.value),
  mean_field = c(mean(cor_field_same$rho, na.rm = TRUE),
                 mean(cor_field_prev$rho, na.rm = TRUE)),
  mean_sim = c(mean(cor_sim_same$rho, na.rm = TRUE),
               mean(cor_sim_prev$rho, na.rm = TRUE))
)

write_csv(alignment_test_results,
          file.path(TABLES_DIR, "alignment_statistical_tests.csv"))
```

5 5. Summary and Conclusions

5.1 5.1 Key Findings Summary

```r
# Create summary table
summary_results <- tribble(
  ~Metric, ~Field_Data, ~Simulated_Data,
  "N Stories", n_distinct(df_field$conversation_id), n_distinct(df_simulated$conversation_id),
  "Mean User Valence", mean(df_field$user_valence, na.rm = TRUE), mean(df_simulated$user_valence, na.rm
  "Mean AI Valence", mean(df_field$ai_valence, na.rm = TRUE), mean(df_simulated$ai_valence, na.rm = TRU
  "Same-Turn Alignment ( )", mean(cor_field_same$rho, na.rm = TRUE), mean(cor_sim_same$rho, na.rm = TRUE
  "Previous-Turn Alignment ( )", mean(cor_field_prev$rho, na.rm = TRUE), mean(cor_sim_prev$rho, na.rm =
) %>%
  mutate(
    Field_Data = round(Field_Data, 3),
    Simulated_Data = round(Simulated_Data, 3)
  )

print(summary_results)
```

```
## # A tibble: 5 x 3
##   Metric                      Field_Data Simulated_Data
##   <chr>                            <dbl>          <dbl>
## 1 N Stories                       82              6
## 2 Mean User Valence               -0.089         -0.407
## 3 Mean AI Valence                 -0.001         -0.296
## 4 Same-Turn Alignment ( )          0.23          -0.246
## 5 Previous-Turn Alignment ( )      0.086          0.533
```

```r
write_csv(summary_results, file.path(TABLES_DIR, "analysis_summary.csv"))
```

6 6. Supplementary: Interaction Effects

6.1 6.1 Starter Interaction Effects

6.1.1 6.1.1 Same-Turn Alignment by Starter

```
lmer_valence_time <- lmer(valence ~ turn + (1 | conversation_id), data = df_field_long)
print(summary(lmer_valence_time)$coefficients)
```

```
##                 Estimate  Std. Error        df    t value  Pr(>|t|)
## (Intercept) -0.052697523 0.032666463  214.9999 -1.6131996 0.1081679
## turn         0.001247722 0.003415634 1393.0000  0.3652974 0.7149449
```

```
# Mixed model with starter interaction
model_starter_same <- lmer(ai_valence ~ user_valence * starter + (1 | conversation_id),
                           data = df_field)

cat("=== Same-Turn Alignment: Starter Interaction ===\n")
```

```
## === Same-Turn Alignment: Starter Interaction ===
```

```
print(summary(model_starter_same)$coefficients)
```

```
##                              Estimate Std. Error        df    t value
## (Intercept)                0.04376871 0.03437129  74.59644  1.2734093
## user_valence               0.33079534 0.04754107 726.28634  6.9580966
## starteruser               -0.04075185 0.04930109  75.02499 -0.8265913
## user_valence:starteruser -0.08403092 0.06831065 733.77813 -1.2301291
##                              Pr(>|t|)
## (Intercept)               2.068267e-01
## user_valence              7.711823e-12
## starteruser               4.110899e-01
## user_valence:starteruser 2.190430e-01
```

6.1.2 6.1.2 Previous-Turn Alignment by Starter

```
# Mixed model with starter interaction
df_field_lag <- df_field %>% filter(!is.na(ai_valence_lag1))

model_starter_prev <- lmer(user_valence ~ ai_valence_lag1 * starter + (1 | conversation_id),
                           data = df_field_lag)

cat("=== Previous-Turn Alignment: Starter Interaction ===\n")
```

```
## === Previous-Turn Alignment: Starter Interaction ===
```

```
print(summary(model_starter_prev)$coefficients)
```

```
##                                 Estimate Std. Error        df     t value
## (Intercept)                -0.099201071 0.03087861  67.70613 -3.21261446
## ai_valence_lag1             0.168011590 0.05514999 631.64691  3.04644821
## starteruser                 0.001528406 0.04422041  67.74380  0.03456337
## ai_valence_lag1:starteruser 0.118965815 0.07897255 591.03559  1.50641977
##                               Pr(>|t|)
## (Intercept)                0.002014578
## ai_valence_lag1            0.002412174
## starteruser                0.972529551
```

```
## ai_valence_lag1:starteruser 0.132493707
```

6.1.3 6.1.3 Per-Story Correlations by Starter

```r
# Same-turn correlations stratified by starter
cor_starter_same <- df_field %>%
  filter(!is.na(starter)) %>%
  group_by(conversation_id, starter) %>%
  filter(n() >= 3) %>%
  summarise(
    rho = cor(user_valence, ai_valence, use = "complete.obs"),
    n_turns = n(),
    .groups = "drop"
  )

cat("=== Same-Turn Correlations by Starter ===\n")
```

```
## === Same-Turn Correlations by Starter ===
```

```r
cor_starter_same %>%
  group_by(starter) %>%
  summarise(
    mean_rho = mean(rho, na.rm = TRUE),
    sd_rho = sd(rho, na.rm = TRUE),
    n_stories = n()
  ) %>%
  print()
```

```
## # A tibble: 2 x 4
##   starter mean_rho sd_rho n_stories
##   <chr>      <dbl>  <dbl>     <int>
## 1 ai         0.342  0.344        42
## 2 user       0.116  0.328        40
```

```r
# Test difference
t_test_starter <- t.test(rho ~ starter, data = cor_starter_same)
cat("\nT-test (starter effect on same-turn rho):\n")
```

```
##
## T-test (starter effect on same-turn rho):
```

```r
print(t_test_starter)
```

```
##
##  Welch Two Sample t-test
##
## data:  rho by starter
## t = 3.0245, df = 78.962, p-value = 0.003359
## alternative hypothesis: true difference in means between group ai and group user is not equal to 0
## 95 percent confidence interval:
##  0.07721825 0.37450162
## sample estimates:
##   mean in group ai mean in group user
##          0.3418585          0.1159986
```

6.2 6.2 LLM Type Interaction Effects

6.2.1 6.2.1 Same-Turn Alignment by LLM Type

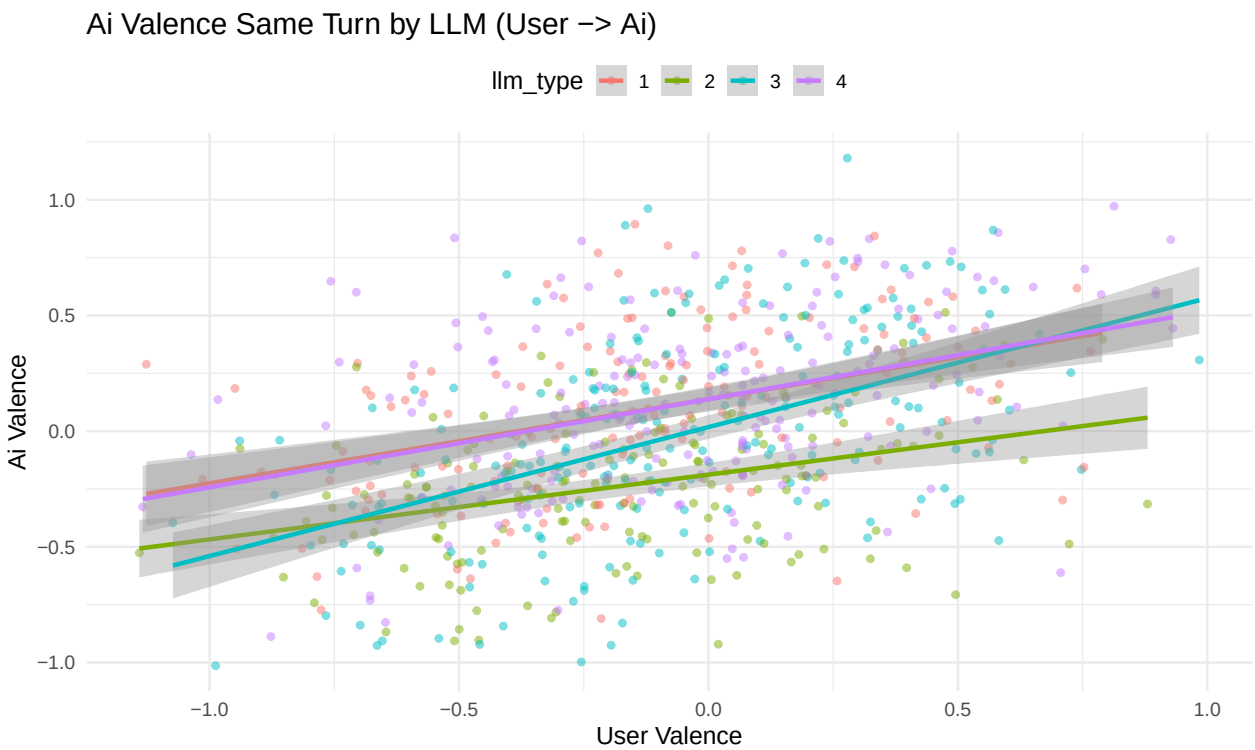
```
# Mixed model with LLM type interaction
df_field_llm <- df_field %>% filter(!is.na(llm_type))

model_llm_same <- lmer(ai_valence ~ user_valence * factor(llm_type) *  + (1 | conversation_id),
                       data = df_field_llm)

cat("=== Same-Turn Alignment: LLM Type Interaction ===\n")
```

```
## === Same-Turn Alignment: LLM Type Interaction ===
```

```
print(summary(model_llm_same)$coefficients)
```

```
##                                    Estimate Std. Error        df     t value
## (Intercept)                     0.129260834 0.04204445  72.70445  3.07438503
## user_valence                    0.286544966 0.06925188 710.15087  4.13772088
## factor(llm_type)2              -0.325091856 0.06023740  76.39695 -5.39684429
## factor(llm_type)3              -0.119795918 0.05533263  71.40734 -2.16501383
## factor(llm_type)4               0.005528522 0.05627928  70.91586  0.09823371
## user_valence:factor(llm_type)2 -0.050024563 0.09867212 727.35358 -0.50697770
## user_valence:factor(llm_type)3  0.136418666 0.09484279 692.76498  1.43836624
## user_valence:factor(llm_type)4 -0.011208209 0.09412313 714.76044 -0.11908029
##                                    Pr(>|t|)
## (Intercept)                     2.970136e-03
## user_valence                    3.928281e-05
## factor(llm_type)2               7.360165e-07
## factor(llm_type)3               3.372969e-02
## factor(llm_type)4               9.220238e-01
## user_valence:factor(llm_type)2 6.123241e-01
## user_valence:factor(llm_type)3 1.507818e-01
## user_valence:factor(llm_type)4 9.052452e-01
```

```
p_llm_same <- df_field_llm %>%
  mutate(llm_type = as.factor(llm_type)) %>%
  ggplot(aes(x = user_valence, y = ai_valence, color = llm_type)) +
  geom_point(alpha = 0.5, size = 1.5) +
  geom_smooth(method = "lm", se = TRUE, size = 1.2) +
  labs(
    title = "Ai Valence Same Turn by LLM (User -> Ai)",
    x = "User Valence",
    y = "Ai Valence"
  ) +
  theme_minimal(base_size = 14) +
  theme(legend.position = "top")

print(p_llm_same)
```




```
ggsave(file.path(GRAPHS_DIR, "alignment_llm_same.png"),
       p_llm_same, width = 12, height = 6, dpi = 300)
```

6.2.2 6.2.2 Previous-Turn Alignment by LLM Type

```
# Mixed model with LLM type interaction
df_field_llm_lag <- df_field %>%
  filter(!is.na(llm_type), !is.na(ai_valence_lag1))

model_llm_prev <- lmer(user_valence ~ ai_valence_lag1 * factor(llm_type) + (1 | conversation_id),
                       data = df_field_llm_lag)

cat("=== Previous-Turn Alignment: LLM Type Interaction ===\n")
```

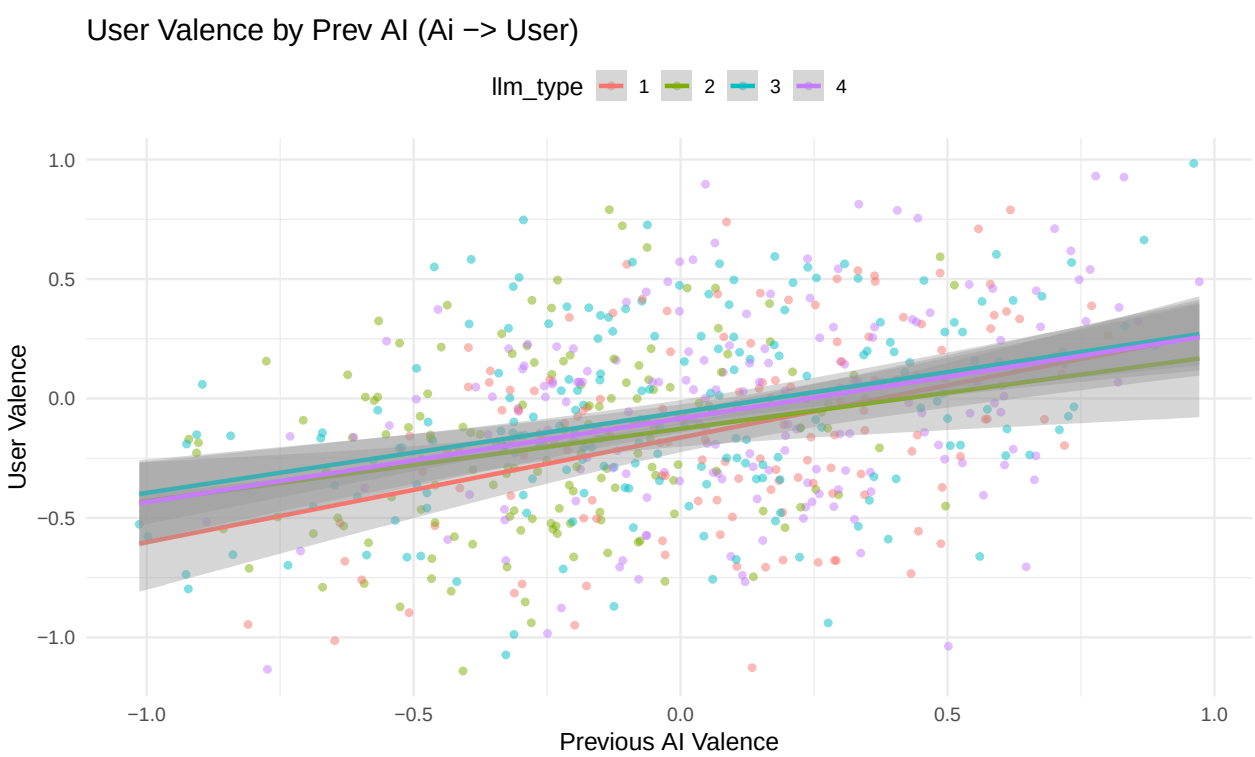
```
## === Previous-Turn Alignment: LLM Type Interaction ===
```

```
print(summary(model_llm_prev)$coefficients)
```

```
##                                      Estimate Std. Error        df     t value
## (Intercept)                       -0.152356802 0.04877387  70.12614 -3.12373835
## ai_valence_lag1                    0.306116495 0.09462310 613.86152  3.23511394
## factor(llm_type)2                  0.002302089 0.07228786  83.75153  0.03184613
## factor(llm_type)3                  0.091555426 0.06415089  68.81278  1.42718861
## factor(llm_type)4                  0.084094740 0.06608875  71.35396  1.27245165
## ai_valence_lag1:factor(llm_type)2 -0.102754208 0.13720888 647.23175 -0.74888890
## ai_valence_lag1:factor(llm_type)3 -0.142545831 0.11601961 612.84058 -1.22863570
## ai_valence_lag1:factor(llm_type)4 -0.079807148 0.12201060 624.10407 -0.65410013
##                                      Pr(>|t|)
## (Intercept)                       0.002595916
## ai_valence_lag1                   0.001281276
## factor(llm_type)2                 0.974670530
```

```
## factor(llm_type)3                 0.158047680
## factor(llm_type)4                 0.207343608
## ai_valence_lag1:factor(llm_type)2 0.454196241
## ai_valence_lag1:factor(llm_type)3 0.219680018
## ai_valence_lag1:factor(llm_type)4 0.513288241
```

```
p_llm_prev <- df_field_llm_lag %>%
  mutate(llm_type = as.factor(llm_type)) %>%
  ggplot(aes(x = ai_valence_lag1, y = user_valence, color = llm_type)) +
  geom_point(alpha = 0.5, size = 1.5) +
  geom_smooth(method = "lm", se = TRUE, fullrange= TRUE, size = 1.2) +
  labs(
    title = "User Valence by Prev AI (Ai -> User)",
    x = "Previous AI Valence",
    y = "User Valence"
  ) +
  theme_minimal(base_size = 14) +
  theme(legend.position = "top")

print(p_llm_prev)
```




```
ggsave(file.path(GRAPHS_DIR, "alignment_llm_prev.png"),
       p_llm_prev, width = 12, height = 6, dpi = 300)
```

6.2.3 6.2.3 Per-Story Correlations by LLM Type

```
# Same-turn correlations stratified by LLM type
cor_llm_same <- df_field %>%
  filter(!is.na(llm_type)) %>%
  group_by(conversation_id, llm_type) %>%
  filter(n() >= 3) %>%
```

```r
  summarise(
    rho = cor(user_valence, ai_valence, use = "complete.obs"),
    n_turns = n(),
    .groups = "drop"
  )

cat("=== Same-Turn Correlations by LLM Type ===\n")
```

```
## === Same-Turn Correlations by LLM Type ===
```

```r
cor_llm_same %>%
  group_by(llm_type) %>%
  summarise(
    mean_rho = mean(rho, na.rm = TRUE),
    sd_rho = sd(rho, na.rm = TRUE),
    n_stories = n()
  ) %>%
  print()
```

```
## # A tibble: 4 x 4
##   llm_type mean_rho sd_rho n_stories
##      <int>    <dbl>  <dbl>     <int>
## 1        1    0.253  0.315        18
## 2        2    0.251  0.389        18
## 3        3    0.243  0.373        24
## 4        4    0.179  0.348        22
```

```r
# ANOVA test
anova_llm <- aov(rho ~ factor(llm_type), data = cor_llm_same)
cat("\nANOVA (LLM type effect on same-turn rho):\n")
```

```
##
## ANOVA (LLM type effect on same-turn rho):
```

```r
print(summary(anova_llm))
```

```
##                  Df Sum Sq Mean Sq F value Pr(>F)
## factor(llm_type)  3  0.077 0.02551   0.199  0.897
## Residuals        77  9.887 0.12840
## 1 observation deleted due to missingness
```

6.3 6.3 Valence Distribution by Conditions

6.3.1 6.3.1 Valence by Starter

```r
df_field %>%
  filter(!is.na(starter)) %>%
  pivot_longer(cols = c(user_valence, ai_valence),
               names_to = "agent", values_to = "valence") %>%
  group_by(starter, agent) %>%
  summarise(
    mean_valence = mean(valence, na.rm = TRUE),
    sd_valence = sd(valence, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  print()
```

```
## # A tibble: 4 x 4
##   starter agent        mean_valence sd_valence
##   <chr>   <chr>               <dbl>      <dbl>
## 1 ai      ai_valence         0.0176      0.393
## 2 ai      user_valence      -0.0792      0.385
## 3 user    ai_valence        -0.0215      0.412
## 4 user    user_valence      -0.0992      0.407
```

6.3.2 6.3.2 Valence by LLM Type

```
df_field %>%
  filter(!is.na(llm_type)) %>%
  pivot_longer(cols = c(user_valence, ai_valence),
               names_to = "agent", values_to = "valence") %>%
  group_by(llm_type, agent) %>%
  summarise(
    mean_valence = mean(valence, na.rm = TRUE),
    sd_valence = sd(valence, na.rm = TRUE),
    n_turns = n(),
    .groups = "drop"
  ) %>%
  print()
```

```
## # A tibble: 8 x 5
##   llm_type agent        mean_valence sd_valence n_turns
##      <int> <chr>               <dbl>      <dbl>   <int>
## 1        1 ai_valence         0.0968      0.363     162
## 2        1 user_valence      -0.113       0.406     162
## 3        2 ai_valence        -0.238       0.312     162
## 4        2 user_valence      -0.177       0.383     162
## 5        3 ai_valence        -0.0153      0.432     216
## 6        3 user_valence      -0.0585      0.386     216
## 7        4 ai_valence         0.126       0.379     198
## 8        4 user_valence      -0.0302      0.395     198
```

6.4 7. Session Information

```
sessionInfo()
```

```
## R version 4.4.2 (2024-10-31)
## Platform: aarch64-apple-darwin20
## Running under: macOS Sequoia 15.6.1
##
## Matrix products: default
## BLAS:   /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/lib/libRblas.0.dylib
## LAPACK: /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/lib/libRlapack.dylib;  LAPACK v
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## time zone: Europe/Copenhagen
## tzcode source: internal
##
## attached base packages:
```

```
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
##  [1] here_1.0.2      afex_1.5-0      cowplot_1.2.0   patchwork_1.3.2
##  [5] zoo_1.8-14      stringi_1.8.7   arrow_22.0.0    lmerTest_3.1-3
##  [9] lme4_1.1-37     Matrix_1.7-1    lubridate_1.9.4 forcats_1.0.1
## [13] stringr_1.6.0   dplyr_1.1.4     purrr_1.2.0     readr_2.1.6
## [17] tidyr_1.3.1     tibble_3.3.0    ggplot2_4.0.1   tidyverse_2.0.0
##
## loaded via a namespace (and not attached):
##  [1] tidyselect_1.2.1    farver_2.1.2        S7_0.2.1
##  [4] fastmap_1.2.0       TH.data_1.1-2       pacman_0.5.1
##  [7] digest_0.6.39       timechange_0.3.0    estimability_1.5.1
## [10] lifecycle_1.0.4     survival_3.7-0      magrittr_2.0.4
## [13] compiler_4.4.2      rlang_1.1.6         tools_4.4.2
## [16] utf8_1.2.6          yaml_2.3.10         knitr_1.50
## [19] labeling_0.4.3      bit_4.6.0           plyr_1.8.9
## [22] RColorBrewer_1.1-3  abind_1.4-8         multcomp_1.4-26
## [25] withr_3.0.2         numDeriv_2016.8-1.1 grid_4.4.2
## [28] xtable_1.8-4        emmeans_2.0.0       scales_1.4.0
## [31] MASS_7.3-61         tinytex_0.58        cli_3.6.5
## [34] mvtnorm_1.3-3       crayon_1.5.3        rmarkdown_2.30
## [37] ragg_1.5.0          reformulas_0.4.2    generics_0.1.4
## [40] rstudioapi_0.17.1   reshape2_1.4.5      tzdb_0.5.0
## [43] minqa_1.2.8         splines_4.4.2       assertthat_0.2.1
## [46] parallel_4.4.2      vctrs_0.6.5         boot_1.3-31
## [49] sandwich_3.1-1      carData_3.0-5       car_3.1-3
## [52] hms_1.1.4           bit64_4.6.0-1       Formula_1.2-5
## [55] systemfonts_1.3.1   glue_1.8.0          nloptr_2.2.1
## [58] codetools_0.2-20    gtable_0.3.6        pillar_1.11.1
## [61] htmltools_0.5.8.1   R6_2.6.1            textshaping_1.0.4
## [64] Rdpack_2.6.4        rprojroot_2.1.1     vroom_1.6.6
## [67] evaluate_1.0.5      lattice_0.22-6      rbibutils_2.4
## [70] Rcpp_1.1.0          coda_0.19-4.1       nlme_3.1-166
## [73] mgcv_1.9-1          xfun_0.54           pkgconfig_2.0.3
```

Analysis complete. All figures saved to `analysis/figures/` and tables to `analysis/tables/`.